Republic of Tunisia
Ministry of Higher Education, Scientific Research
and Information and Communication
Technologies
Sfax University
National Engineering School of Sfax

Computer Engineering & Applied Mathematics
Department

ST-EN07/00
GRADUATION PROJECT
Serial N°: 2015 / DIMA-033

GRADUATION PROJECT REPORT

presented at

National Engineering School of Sfax
(Computer Engineering & Applied Mathematics Department)

in order to obtain the

National Engineering Diploma in Computer Science

by

Oumayma JELLALI

Secure AI Assistant

Defended on 12/12/1234 in front of the committee composed of

Mr Foulen FOULENI	President
Mr Mohamed JMAIEL	Supervisor
Mr Foulen FOULENI	Reviewer

Dedication

Put your dedication lines here.

And try to be expressive ;)

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum

To all of you,
I dedicate this work.

Oumayma JELLALI

Thanks

And put your thanks here.

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Contents

Dedication **i**

Thanks **ii**

Contents **iv**

List of Figures **v**

Acronyms **vi**

Introduction **1**

1 Context and State of the Art **2**

1.1 Host Organization Presentation 2

1.1.1 Career Policy and Professional Development 2

1.1.2 Services 3

1.2 Project Context 4

1.2.1 Motivation 4

1.2.2 Problematic 4

1.2.3 Proposed Solution 5

1.2.4 Project Objectives 6

1.3 Study of the Existing Solutions 7

1.3.1 Existing Commercial Solutions 7

1.3.2 Comparative Study 8

1.3.3 Critique and Limitations 9

1.4 Theoretical Background and Key Concepts 9

1.4.1 Large Language Models 9

1.4.2 Retrieval-Augmented Generation 10
1.4.3 Vector Embeddings and Semantic Search 10
1.4.4 Hybrid Retrieval and Reranking 11
1.4.5 Permission-Based Access Control 11
1.5 Project Management 12
1.5.1 Work Methodology 12
1.5.2 Project Planning Tool 14
1.5.3 Project Planning 14

2 Requirements Analysis and Specification 16
2.1 Requirements Analysis 16
2.1.1 Functional Requirements 16
2.1.2 Non-Functional Requirements 18
2.2 System Actors 19
2.2.1 Actor Identification 19
2.2.2 Actor Description 19
2.3 Use Case Diagram 20
2.3.1 Global Use Case Diagram 20
2.4 Sequence Diagrams 21
2.4.1 Document Ingestion Sequence 21
2.4.2 Query and Answer Generation Sequence 22

Conclusion 24

Appendix 25

Webography 26

Bibliography 27

List of Figures

1 Consultim-IT Logo 2
2 CRISP-DM methodology 13
3 Project Gantt diagram (February – June 2025) 15
4 Global use case diagram 20
5 Document ingestion sequence diagram 21
6 Query and answer generation sequence diagram 22

Acronyms

AI Artificial Intelligence

API Application Programming Interface

BGE-M3 BAAI General Embedding Multilingual Model

CRISP-DM Cross-Industry Standard Process for Data Mining

ENIS National Schoole of Engineering of Sfax

LLM Large Language Model

PDF Portable Document Format

RAG Retrieval-Augmented Generation

RBAC Role-Based Access Control

RRF Reciprocal Rank Fusion

SaaS Software as a Service

Introduction

Welcome to National Schoole of Engineering of Sfax (ENIS).

Again, welcome to ENIS.

Your introduction goes here.

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Chapter 1

Context and State of the Art

This chapter introduces the context in which this project was carried out. It presents the host organization, defines the problem being addressed, examines existing solutions and their limitations, covers the theoretical foundations the system relies on, and closes with an overview of the project management approach adopted throughout the internship.

1.1 *Host Organization Presentation*

Consultim-IT is an IT consulting and engineering company that helps businesses think through and build their digital solutions. The company works almost exclusively within the **Microsoft ecosystem Azure**, **.NET**, **SharePoint**, **Power BI**, and **Microsoft 365** and serves clients across sectors like banking, telecoms, manufacturing, the public sector, and academia.

Figure 1. *Consultim-IT Logo [1]*

1.1.1 Career Policy and Professional Development

One thing that genuinely stands out about Consultim-IT is how much it invests in its own people. Consultants get hands-on mentorship, access to technical training, and real support for earning professional certifications. The idea is simple: the quality of a consulting firm depends entirely

on the expertise of its consultants, so keeping that expertise sharp isn't just encouraged it's built into how the company operates.

1.1.2 Services

The company covers the full spectrum of IT consulting and engineering. Its main service areas are:

- **Business Analysis**

  Before writing a single line of code, consultants sit down with the client to understand the real problem mapping processes, spotting pain points, and making sure the solution actually fits the business.

- **Software Development**

  From web and mobile apps to cloud migrations and **SharePoint**-based platforms, the team builds solutions tailored to each client's specific needs.

- **Functional Analysis**

  Translating business requirements into concrete system specifications defining features, designing interfaces, and keeping development teams aligned throughout the project.

- **Training and Change Management**

  Rolling out new technology is only half the job. Consultim-IT also handles user training and change management to make sure people actually adopt what's been built.

- **Quality Assurance**

  Testing, validating, and documenting everything before it goes live so what gets delivered actually works.

- **Artificial Intelligence and Advanced Data Analysis**

  Helping clients make real use of their data through predictive models, machine learning integration, and Artificial Intelligence (AI)-driven process automation.

- **Cybersecurity and Information Systems Protection**

  Risk audits, access management, cloud security, and threat monitoring because no serious digital project can afford to ignore this anymore.

1.2 *Project Context*

1.2.1 Motivation

Anyone who has worked in a consulting firm knows the feeling: you are fairly certain that a colleague already wrote something relevant to the question in front of you a report, an email thread, a slide deck but finding it means digging through SharePoint folders, searching Gmail, and opening a dozen files before giving up and starting from scratch.

This is not a productivity complaint. It is a structural problem. Organizations accumulate knowledge constantly, but that knowledge ends up scattered across disconnected platforms with no common way to query it. The information exists it is just practically unreachable.

At Consultim-IT, consultants work daily across **SharePoint** repositories, **Gmail** inboxes, and a wide variety of document formats. There is currently no intelligent interface that lets them ask a plain question and get a precise, sourced answer drawn from all of those sources at once. Cloud-based AI tools could partially fill that gap, but sending client documents to an external service is simply not an option in a consulting environment where confidentiality is a baseline requirement.

That is the gap this project sets out to fill.

1.2.2 Problematic

Building an assistant that actually solves this problem is harder than it sounds. Several challenges have to be addressed at once:

- **Data privacy and local deployment**

  Sending enterprise documents to external AI Application Programming Interfaces (APIs) OpenAI, Google, or otherwise is not acceptable when confidentiality agreements are in place. The entire pipeline, from ingestion to answer generation, has to run locally. No data should leave the organization's infrastructure, ever.

- **Knowledge spread across too many formats**

  Enterprise knowledge does not sit neatly in one place. It lives in Portable Document Format (PDF) reports, Word documents, PowerPoint decks, scanned files, spreadsheets, emails, and images each with its own structure. A system that only handles plain text will miss most of what actually matters.

- **Access control that actually works**

  Not everyone should see everything. A junior consultant should not be able to query documents restricted to management, and the system should not surface confidential reports to someone who has no business reading them. Permission enforcement has to happen at retrieval time, not just at the login screen.

- **Answers that can be trusted**

  Language models are good at sounding confident even when they are wrong. In an enterprise setting, a wrong answer delivered with authority can cause real damage. Every response the system produces has to be grounded in actual retrieved documents, with citations that let the user verify the source.

- **Understanding documents beyond plain text**

  Tables carry data that cannot be treated as prose. Scanned files have no machine-readable text at all. Emails arrive with attachments of all kinds. Meaningful retrieval requires understanding documents at a structural level, not just extracting raw characters and hoping for the best.

1.2.3 Proposed Solution

The response to these challenges is a **secure, fully local Retrieval-Augmented Generation (RAG)-based AI assistant** that can query heterogeneous enterprise knowledge sources while respecting both data privacy and user permissions.

The system is built around four core ideas:

- **Everything stays local**

  Document processing, embedding, vector storage, and answer generation all run

on-premise using open-source models. The language model runs locally via **Ollama**. Nothing is sent to an external service.

- **Connectors for real enterprise sources**

  The system connects directly to **SharePoint** and **Gmail**, and ingests local files across seven formats: PDF (digital and scanned), DOCX, PPTX, EML, TXT, CSV, and images. Each format goes through a dedicated handler that preserves structure tables, sections, captions rather than flattening everything into raw text.

- **Permissions enforced at retrieval time**

  Every document is tagged with a unique identifier and tied to the permissions of the user who uploaded it. When a query comes in, the retrieval layer filters candidates against what that specific user is actually allowed to see. The access control happens inside the pipeline, not just at the door.

- **A full RAG pipeline with quality control**

  Queries go through hybrid dense and sparse retrieval over a **Qdrant** vector database, cross-encoder reranking, and grounded answer generation with inline source citations. A **LangGraph** orchestration layer handles query classification, rewriting, chunk quality grading, and automatic retry when retrieval results are not good enough.

1.2.4 Project Objectives

More concretely, this project aims to:

- Design and implement a modular ingestion pipeline that handles at least seven file formats, including scanned PDFs and email attachments with nested files.
- Build a hybrid retrieval system combining dense semantic search and sparse keyword search with Reciprocal Rank Fusion (RRF) fusion, with measurably better results than either approach alone.
- Enforce document-level access control at query time so that users only ever get answers drawn from content they are authorized to see.

- Implement source-grounded answer generation with hallucination detection and automatic retry on weak retrieval.
- Connect to live **SharePoint** and **Gmail** sources so the assistant can query real, up-to-date enterprise data.
- Expose everything through a **FastAPI** backend with multi-conversation memory, user authentication, and a streaming chat interface.

1.3 *Study of the Existing Solutions*

Before building anything, it is worth asking whether something already exists that solves this problem. Two platforms stand out as the most relevant and technically serious alternatives in the enterprise AI assistant space. Open-source alternatives such as PrivateGPT [2] and AnythingLLM [3] exist but remain limited to single-user setups with no enterprise connectors or permission enforcement, making them unsuitable for a multi-user consulting environment.

1.3.1 Existing Commercial Solutions

1.3.1.1 Microsoft 365 Copilot

Microsoft 365 Copilot is the most natural point of comparison for this project. It is built directly into the Microsoft 365 ecosystem and can query across SharePoint, Teams, Outlook, and other Microsoft services through the Microsoft 365 Graph API. Access control is handled via Microsoft Purview, which inherits the existing SharePoint and Entra ID permission structure so users only retrieve content they are already authorized to see [4].

The problem is straightforward: Copilot is a cloud-based service.It requires a permanent connection to Microsoft's infrastructure and cannot run on-premise or in air-gapped environments [4]. For organizations that handle sensitive client data under confidentiality agreements, this is not a configuration issue it is a fundamental incompatibility.

1.3.1.2 Glean

Glean is the most technically serious competitor in this space. It connects to over 100 enterprise tools Jira, Confluence, Box, Slack, and many others and maintains real-time permission synchronization that mirrors the Role-Based Access Control (RBAC) of each source system [5]. If a user loses access to a document in Confluence, that change propagates to Glean immediately. Its security architecture follows a zero-trust model with single-tenant infrastructure [5], which is a genuinely serious approach to data isolation.

But Glean is a Software as a Service (SaaS) platform. There is no on-premise deployment option, no self-hosted edition, and no way to run it inside a private infrastructure [5]. For organizations with data sovereignty requirements, that closes the conversation.

1.3.2 Comparative Study

The table below summarizes the key criteria across both solutions alongside the system proposed in this project:

Table 1. *Comparison of existing solutions with the proposed system*

Criterion	**MS 365 Copilot**	**Glean**	**Proposed System**
Local / On-premise deployment	×	×	✓
Permission-based access control	✓	✓	✓
Multi-format document support	Partial	✓	✓
Multi-source connectors	✓	✓	✓
Full data sovereignty	×	×	✓
Open source / Customizable	×	×	✓
Free of vendor dependency	×	×	✓

1.3.3 Critique and Limitations

Both solutions are well-engineered and solve real problems but they are built for organizations that are comfortable putting their data in the cloud. Copilot is locked to Microsoft's infrastructure. Glean is a SaaS platform with no local deployment path. Neither gives an organization full control over where its data goes or how the pipeline works under the hood.

This is not a minor limitation. In a consulting environment where client confidentiality is a contractual obligation, sending documents to an external service even a secure one is simply not an option.

The system proposed in this project is not trying to compete with Copilot or Glean on features. It is answering a different question: what does an enterprise AI assistant look like when the data cannot leave the building?

1.4 *Theoretical Background and Key Concepts*

Understanding how the proposed system works requires a grasp of a few core concepts. This section introduces them in the order they appear in the pipeline from how documents are represented mathematically, to how answers are generated from retrieved content.

1.4.1 Large Language Models

A Large Language Model (LLM) is a neural network trained on massive amounts of text data to understand and generate human language. The key idea behind modern LLMs is the transformer architecture, introduced by Vaswani et al. in 2017 [6], which allows the model to weigh the relationships between all words in a sequence simultaneously rather than processing them one by one.

In practice, a LLM takes a piece of text as input a question, a document, an instruction and produces a continuation of that text as output. What makes recent models genuinely useful is

not just their size but their ability to follow instructions, reason across context, and adapt their tone and format to what is asked of them.

For this project, the language model runs entirely locally via **Ollama** [7], using open-source models that require no connection to an external API. This is what makes local deployment possible without sacrificing generation quality.

1.4.2 Retrieval-Augmented Generation

LLMs have a fundamental limitation: they only know what was in their training data. Ask a model about a document it has never seen, and it will either hallucinate an answer or admit it does not know. RAG solves this by giving the model access to external knowledge at query time rather than baking everything into the weights [8].

The idea is simple. When a user asks a question:

1. The question is used to search a knowledge base and retrieve the most relevant document chunks.
2. Those chunks are injected into the model's context window alongside the question.
3. The model generates an answer grounded strictly in the retrieved content, with source citations.

This approach keeps the model honest it can only answer from what was actually retrieved and makes the system updatable without retraining. Adding a new document to the knowledge base is enough to make its content immediately queryable [8].

1.4.3 Vector Embeddings and Semantic Search

To retrieve relevant documents, the system needs a way to measure semantic similarity between a question and thousands of document chunks. This is where vector embeddings come in.

An embedding model converts any piece of text into a dense numerical vector a point in a high-dimensional space such that semantically similar texts end up close together [9].

A question about *data privacy policies* will land near chunks that discuss *confidentiality agreements* and *access control*, even if they share no exact words.

In this project, embeddings are produced by **BGE-M3** [10], a multilingual embedding model developed by BAAI that supports both dense and sparse vector representations simultaneously. Dense vectors capture semantic meaning; sparse vectors capture exact keyword matches. Using both together is what makes hybrid retrieval possible.

1.4.4 Hybrid Retrieval and Reranking

No single retrieval strategy works best in all situations. Dense semantic search is good at finding conceptually related content even when the exact words differ, but it can miss precise technical terms, identifiers, or model names. Sparse keyword search handles exact matches well but fails on paraphrased or conceptual queries.

Hybrid retrieval combines both approaches. The system runs a dense search and a sparse search in parallel, then merges the two ranked lists using RRF [11]. RRF assigns a score to each document based on its rank in each list rather than its raw similarity score, which makes the fusion robust to score scale differences between the two methods.

The merged list is then passed to a cross-encoder reranker **BGE-reranker-v2-m3** [10] which scores each candidate chunk against the query jointly, rather than independently. This is significantly more accurate than embedding-based similarity but too slow to run on thousands of candidates, which is why it is applied only to the top results after the initial retrieval step.

1.4.5 Permission-Based Access Control

Access control in AI systems is more subtle than it sounds. Enforcing permissions at the interface level showing or hiding certain menu options is not enough. If the retrieval layer can surface content from documents the user is not authorized to see, the system has a security problem regardless of what the interface shows.

In this project, access control is enforced inside the retrieval pipeline itself. Every document chunk stored in the **Qdrant** vector database [12] carries a unique document identifier linked to the permissions of the user who uploaded it. At query time, the retrieval layer filters candidate chunks against the querying user's allowed document set before any ranking or generation happens. A user cannot receive an answer drawn from a document they are not permitted to access not because the interface hides it, but because it never enters the retrieval pool in the first place.

1.5 *Project Management*

1.5.1 Work Methodology

Managing an AI pipeline project is different from managing a classic software development project. The work is inherently iterative you do not know exactly what your retrieval strategy will look like until you have processed real documents, and you do not know how well your generation works until you have a retrieval layer to feed it. A linear methodology would have been a poor fit.

For this reason, we adopted **CRISP-DM** (Cross-Industry Standard Process for Data Mining) [13] as the guiding methodology. CRISP-DM is a widely used framework in data science and AI projects that structures work into six iterative phases, allowing the team to revisit earlier stages as understanding deepens.

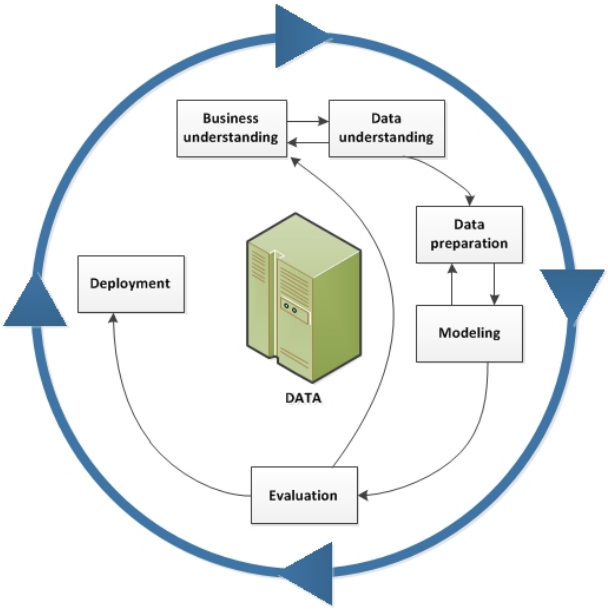

Figure 2. *The CRISP-DM methodology [13]*

The six phases of CRISP-DM mapped naturally onto this project as follows:

- **Business Understanding**

  Understanding the real problem: enterprise knowledge scattered across disconnected platforms, no local AI assistant, and strict confidentiality constraints. This phase produced the project context, the study of existing solutions, and the definition of concrete objectives.

- **Data Understanding**

  Analyzing the nature of enterprise data the variety of file formats, the structure of SharePoint repositories and Gmail inboxes, and the challenges posed by scanned documents, nested email attachments, and heterogeneous content types.

- **Data Preparation**

  Designing and implementing the full document ingestion pipeline: file handlers for seven formats, semantic chunking strategies, content cleaning, and BAAI General Embedding Multilingual Model (BGE-M3) embedding into the Qdrant vector database.

- **Modeling**

  Building the RAG pipeline: hybrid dense and sparse retrieval with RRF fusion,

cross-encoder reranking, LangGraph orchestration with query classification and retry logic, and the four-layer memory system.

- **Evaluation**

  Assessing the quality of the system using an LLM-as-a-judge framework that scores faithfulness, context relevance, and answer relevance for each query.

- **Deployment**

  Exposing the system through a **FastAPI** backend with user authentication, multi-conversation memory, document upload with permission tagging, SharePoint and Gmail connectors, and a streaming chat interface.

1.5.2 Project Planning Tool

Task tracking and sprint organization were managed using **Trello** [14], a Kanban-based project management tool that allows tasks to be organized into columns representing their current state. Throughout the project, the board was structured around four columns: *Backlog*, *In Progress*, *Review*, and *Done*. Each card represented a specific development task a file handler, a chunking strategy, a retrieval component, or an API endpoint and moved across columns as work progressed.

This lightweight approach suited the iterative nature of Cross-Industry Standard Process for Data Mining (CRISP-DM) well: new tasks emerged naturally as earlier phases revealed new requirements, and the board provided a clear view of what was in progress at any given point without the overhead of formal sprint planning.

1.5.3 Project Planning

The project ran from **February 1, 2025** to **June 1, 2025**, covering four months of development organized across eight phases. The timeline below shows how the phases were distributed across the project duration.

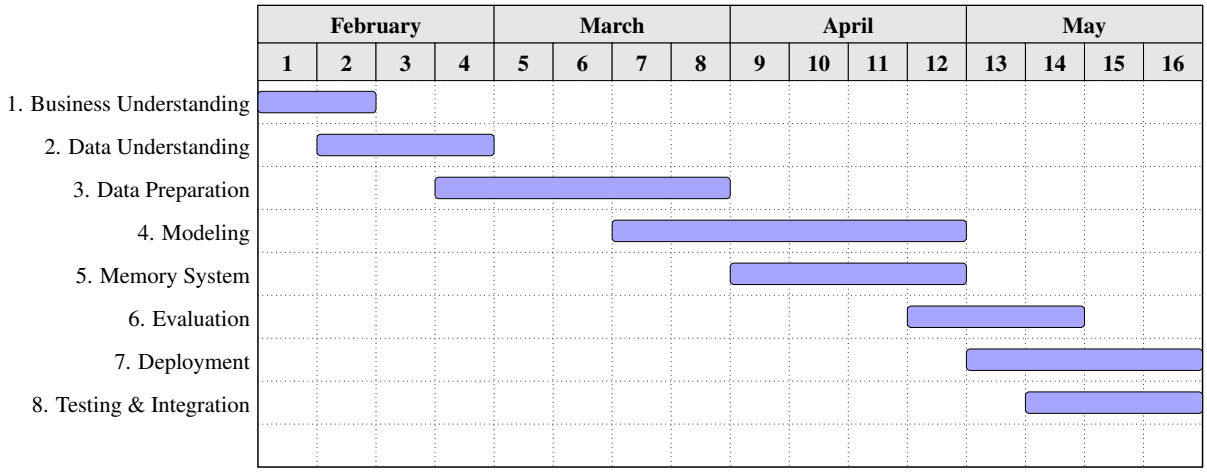

Figure 3. *Project Gantt diagram (February – June 2025)*

Conclusion

This chapter established the foundation of the project. The problem is clear, the limitations of existing solutions are documented, and the theoretical concepts behind the proposed system have been introduced. The next chapter formalizes the system requirements and specifies the expected behavior through use case and sequence diagrams.

Chapter 2

Requirements Analysis and Specification

This chapter defines what the system needs to do before any architecture or implementation decision is made. It identifies the actors who interact with the system, specifies the functional and non-functional requirements, and formalizes the expected behavior through use case and sequence diagrams.

2.1 *Requirements Analysis*

2.1.1 Functional Requirements

Functional requirements describe what the system must be capable of doing. Based on the project objectives defined in Chapter 1, the following requirements were identified:

Table 2. *Functional requirements*

ID	Requirement	Description
FR01	Document Ingestion	The system must ingest documents in at least seven formats: PDF (digital and scanned), DOCX, PPTX, EML, TXT, CSV, and images.
FR02	Structure Preservation	The system must extract and preserve document structure including tables, sections, captions, and email attachments.
FR03	Document Embedding	The system must embed document chunks into dense and sparse vector representations for hybrid retrieval.
FR04	Hybrid Retrieval	The system must retrieve relevant chunks using a combination of dense semantic search and sparse keyword search with RRF fusion.
FR05	Permission Filtering	The system must filter retrieved chunks at query time based on the querying user's document access permissions.
FR06	Answer Generation	The system must generate answers grounded strictly in retrieved documents, with inline source citations.
FR07	Conversation Memory	The system must maintain multi-turn conversation context through a four-layer memory system.
FR08	Document Management	The system must allow users to upload, list, and delete documents, with automatic permission tagging on upload.
FR09	Enterprise Connectors	The system must connect to **SharePoint** and **Gmail** to ingest live enterprise data.
FR10	Streaming Interface	The system must stream answers token by token to the user interface in real time.
FR11	Multi-User Support	The system must support multiple simultaneous users with isolated conversation histories and independent permission sets.
FR12	Authentication	The system must authenticate users before granting access to any functionality.

2.1.2 Non-Functional Requirements

Non-functional requirements define the quality constraints the system must satisfy beyond its core functionality:

Table 3. *Non-functional requirements*

ID	Requirement	Description
NFR01	Security	All data must remain local. No document content, query, or answer may be transmitted to an external service or API.
NFR02	Performance	The system must stream the first answer token to the user within a reasonable delay after a query is submitted, without perceptible blocking.
NFR03	Reliability	The system must automatically retry retrieval with an alternative strategy when chunk quality grading detects weak results.
NFR04	Scalability	The vector database must handle a growing document base without requiring architectural changes.
NFR05	Maintainability	The pipeline must be modular each component (handler, chunker, embedder, retriever, generator) must be independently replaceable without affecting the others.
NFR06	Traceability	Every answer produced by the system must include citations identifying the source document and page number.
NFR07	Confidentiality	The permission system must guarantee that a user can never receive an answer drawn from a document they are not authorized to access.

2.2 *System Actors*

2.2.1 Actor Identification

Three actors interact with the system, each with a distinct role and a different level of interaction:

2.2.2 Actor Description

Table 4. *System actors*

Actor	**Type**	**Description**
User	Human	A consultant or employee who registers, authenticates, uploads documents, asks questions in natural language, and manages their conversation history. The system returns answers drawn exclusively from documents the user is authorized to access.
Administrator	Human	A privileged user who manages user accounts, grants or revokes document access permissions, and oversees the document base. The administrator has access to all documents regardless of who uploaded them.
External System	Automated	Represents the **SharePoint** and **Gmail** connectors that feed live enterprise data into the ingestion pipeline automatically, without requiring manual file uploads from the user.

2.3 *Use Case Diagram*

2.3.1 Global Use Case Diagram

The following diagram presents all interactions between the system actors and the main system functionalities. It gives a high-level view of what each actor can do and how the system boundaries are defined.

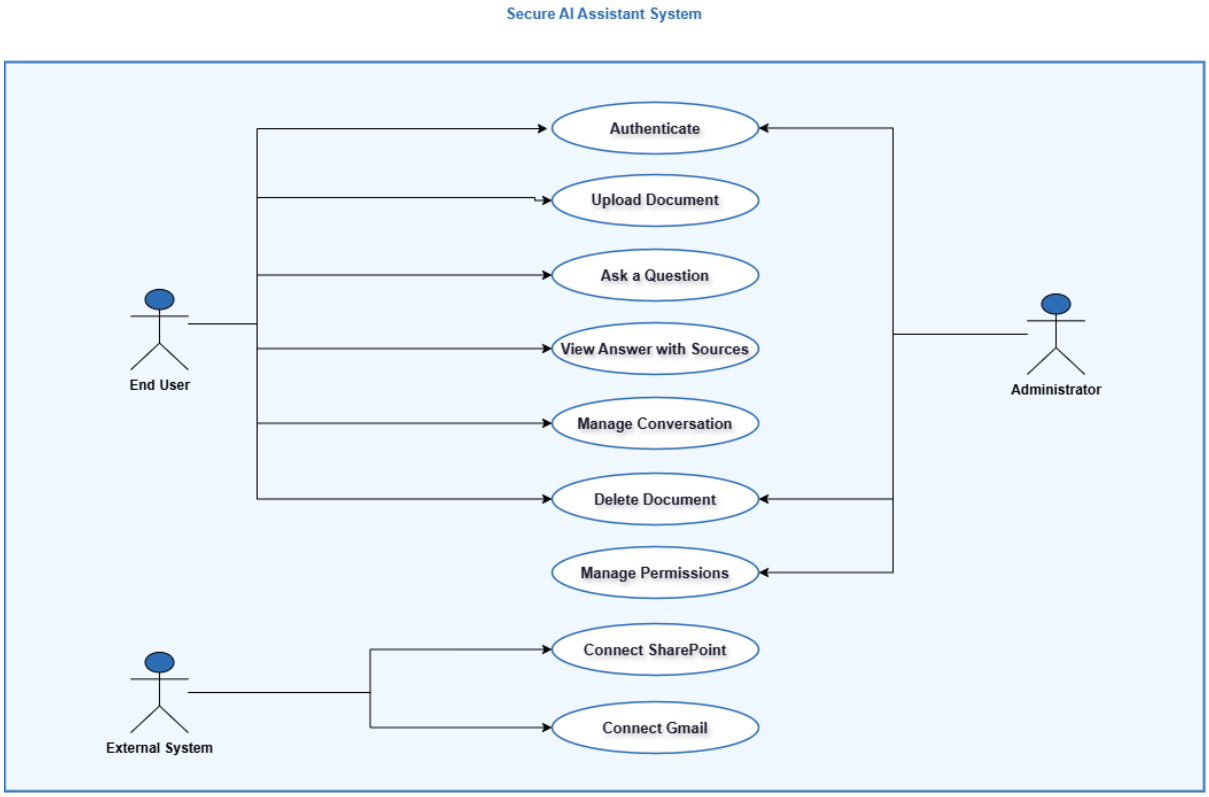

Figure 4. *Global use case diagram*

The **User** is the primary actor. They authenticate, upload documents, query the system in natural language, and manage their conversation history. The **Administrator** is a privileged actor who manages user accounts and controls document access permissions. The **External System** actor represents the automated connectors that bring live data from **SharePoint** and **Gmail** into the ingestion pipeline without any manual intervention from the user.

2.4 *Sequence Diagrams*

Sequence diagrams describe the dynamic behavior of the system by showing the interactions between components over time. Two core flows are formalized here: document ingestion and query processing.

2.4.1 Document Ingestion Sequence

When a user uploads a document, the system triggers a multi-step pipeline that processes, cleans, chunks, embeds, and indexes the content before returning a confirmation. The diagram below captures this flow from the moment the file reaches the API to the moment it is stored in the vector database.

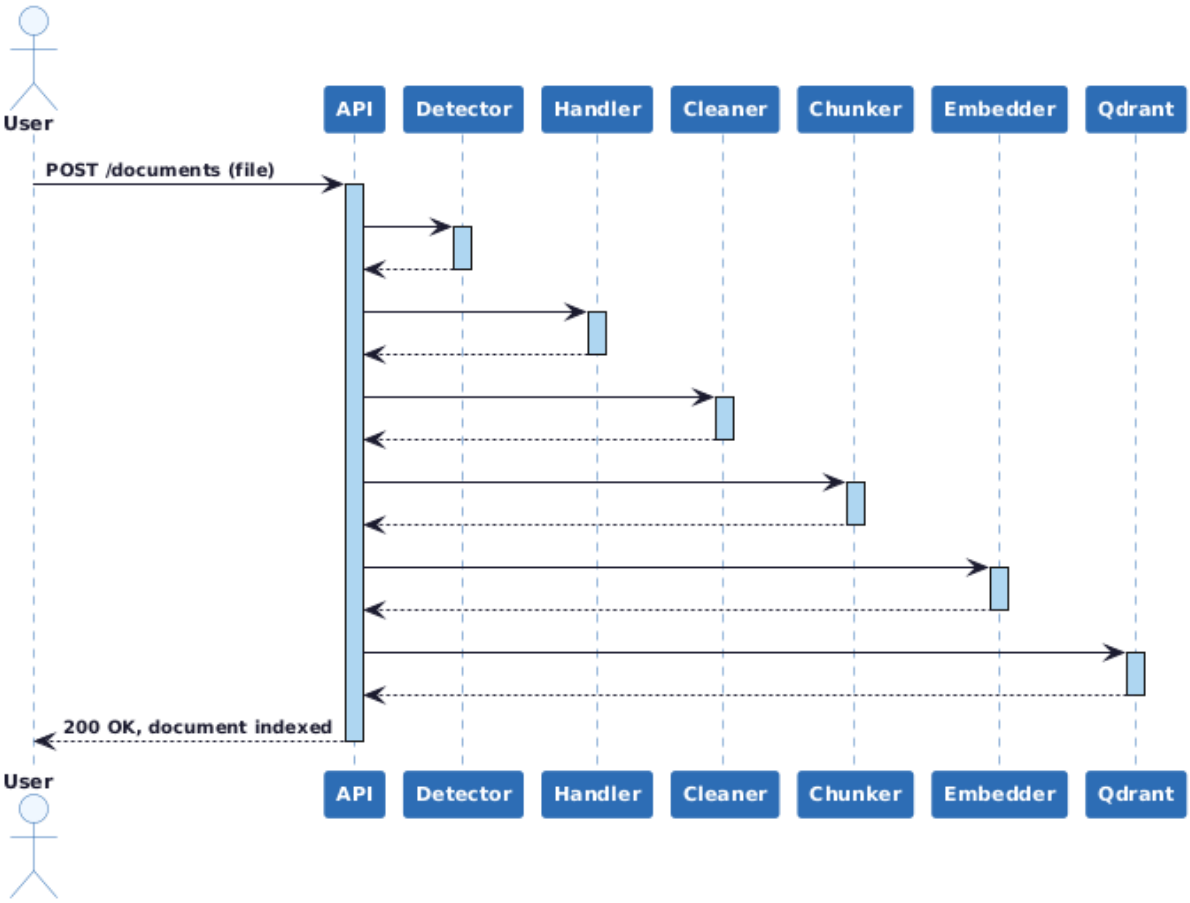

Figure 5. *Document ingestion sequence diagram*

The flow is strictly sequential and synchronous each step must complete successfully before the next one begins. If any step fails, the pipeline stops and an error is returned to the user. No partial ingestion is stored in the vector database.

2.4.2 Query and Answer Generation Sequence

When a user submits a question, the system first retrieves the conversation history from memory, then classifies the query to determine whether retrieval is needed. If the classifier decides the query is conversational (*DIRECT_ANSWER*), the system responds immediately without touching the vector database. If the query requires document retrieval (*NEEDS_RAG*), the full pipeline is activated: retrieval, reranking, generation, and memory update.

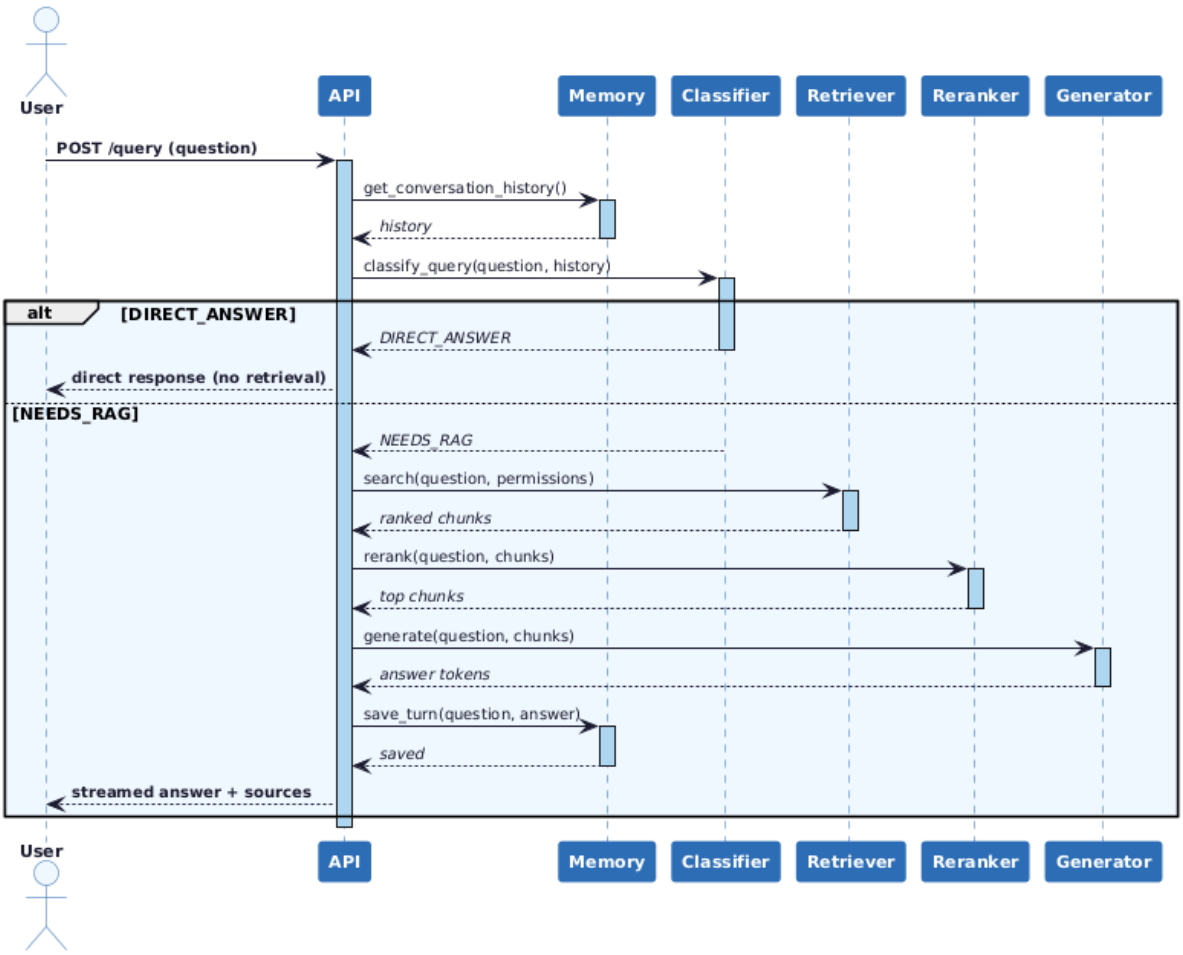

Figure 6. *Query and answer generation sequence diagram*

The branching logic visible in the diagram reflects a deliberate design choice: not every question needs to go through the full RAG pipeline. Routing conversational queries directly to the language model reduces latency and avoids unnecessary retrieval operations on simple follow-up questions or greetings.

Conclusion

This chapter formalized what the system needs to do. The functional and non-functional requirements define the boundaries of the system, the actor table identifies who interacts with it, and the use case and sequence diagrams describe the expected behavior at both a high level and a detailed flow level. The next chapter translates these specifications into a concrete architecture and technical implementation.

Conclusion

And a very interesting conclusion here.

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Appendix

An appedix if you need it.

Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Webography

[1] *Consultim-IT IT Consulting and Engineering*. URL: https://consultim-it.com/ (visited on 0005–2026).

[2] Zylon. *PrivateGPT — Ask questions to your documents without an internet connection*. URL: https://github.com/zylon-ai/private-gpt (visited on 0005–2026).

[3] Mintplex Labs. *AnythingLLM — The all-in-one AI application*. URL: https://github.com/Mintplex-Labs/anything-llm (visited on 0005–2026).

[4] Microsoft. *Microsoft 365 Copilot Documentation*. URL: https://learn.microsoft.com/en-us/microsoft-365/copilot/ (visited on 0005–2026).

[5] Glean. *Glean Security*. URL: https://www.glean.com/security (visited on 0005–2026).

[7] Ollama. *Ollama — Get up and running with large language models locally*. URL: https://ollama.com (visited on 0005–2026).

[10] Jianlyu Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. URL: https://arxiv.org/abs/2309.07597 (visited on 0005–2026).

[12] Qdrant. *Qdrant — Vector Database for the next generation of AI applications*. URL: https://qdrant.tech (visited on 0005–2026).

[13] IBM. *CRISP-DM: A Standard Process for Data Mining*. URL: https://www.ibm.com/docs/en/spss-modeler/18.4.0?topic=dm-crisp-help-overview (visited on 0005–2026).

[14] Atlassian. *Trello — Manage Your Team's Projects From Anywhere*. URL: https://trello.com (visited on 0005–2026).

[15] *Latex @ Wikipedia*. URL: https://en.wikipedia.org/wiki/LaTeX (visited on 0004–2016).

[16] *ENIS*. URL: http://www.enis.rnu.tn/site/enis_fr/ (visited on 0004–2016).

Bibliography

[6] Ashish Vaswani et al. "Attention is All You Need". In: *Advances in Neural Information Processing Systems* 30 (2017). URL: https://arxiv.org/abs/1706.03762.

[8] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems* 33 (2020). URL: https://arxiv.org/abs/2005.11401.

[9] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (2019). URL: https://arxiv.org/abs/1908.10084.

[11] Gordon V Cormack, Charles L A Clarke, and Stefan Buettcher. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods". In: *Proceedings of the 32nd International ACM SIGIR Conference* (2009). URL: https://dl.acm.org/doi/10.1145/1571941.1572114.

[17] Charles Bazerman et al. *Shaping written knowledge: The genre and activity of the experimental article in science*. Vol. 356. University of Wisconsin Press Madison, 1988.

[18] Ashraf Aboulnaga, Alaa R Alameldeen, and Jeffrey F Naughton. "Estimating the selectivity of XML path expressions for internet scale applications". In: *VLDB*. Vol. 1. 2001, pp. 591–600.

[19] Ashbindu Singh. “Review article digital change detection techniques using remotely-sensed data”. In: *International journal of remote sensing* 10.6 (1989), pp. 989–1003.

Hello World!

Here is my test PDF document.